

48730/32548 Cyber Security

Lab 3 – Week 4

Cryptography – SSL – X.509 Certificates

RISHIKESH VIJAY

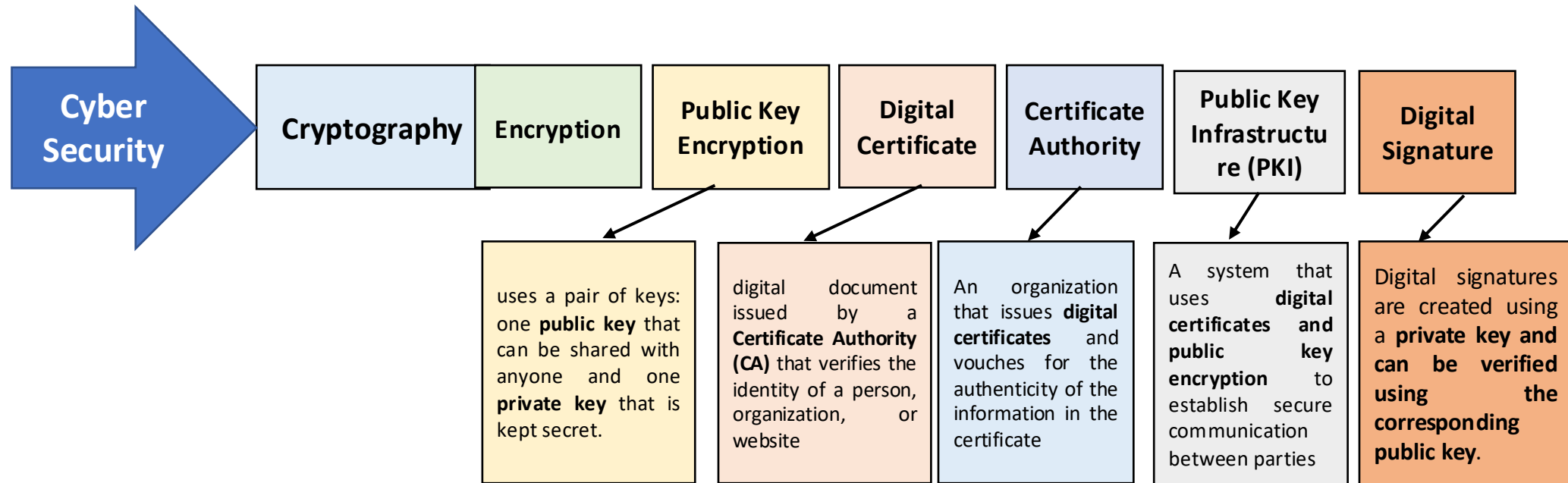
Rishikesh.Vijay-1@uts.edu.au

LAB OBJECTIVE

- The main objective for this lab is to **gain first-hand experience on applications of cryptography** in cyber security and to get familiar with the concepts in the **Public-Key encryption** and **Public-Key Infrastructure (PKI)**

Note: For this lab, only CyberSec-Server VM is required

LAB OVERVIEW



Importance of Cryptography

The goals are to:

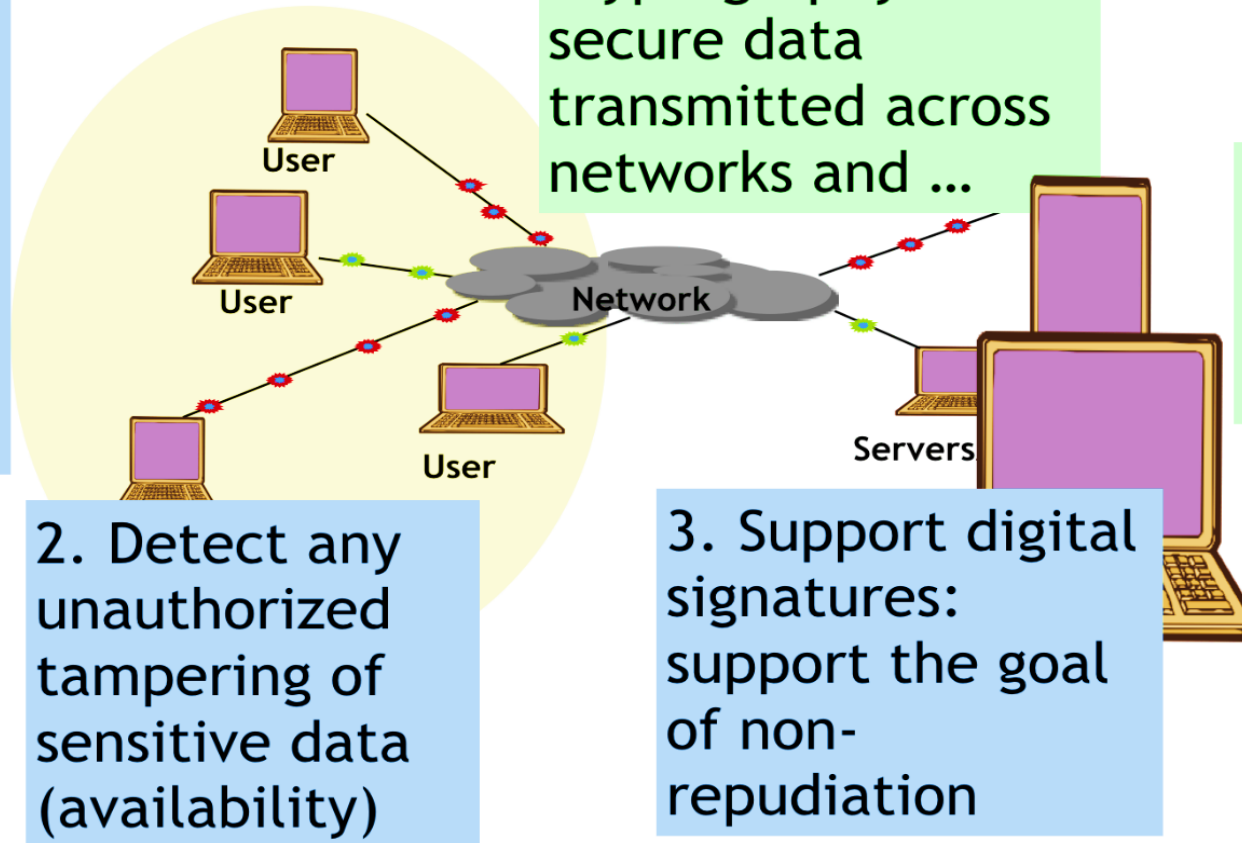
1. Prevent sensitive data from being read without authorization (confidentiality)

2. Detect any unauthorized tampering of sensitive data (availability)

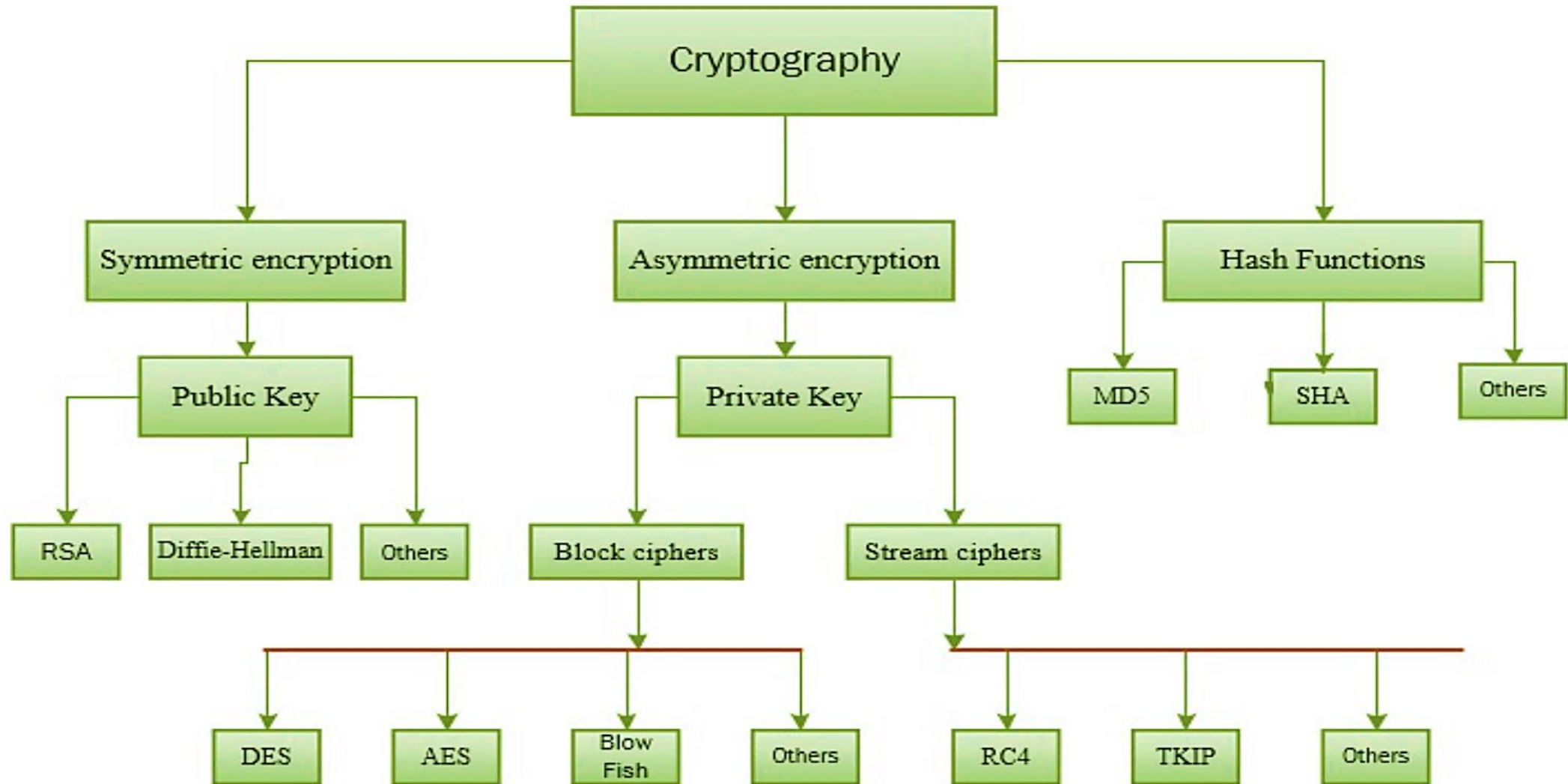
3. Support digital signatures: support the goal of non-repudiation

We study cryptography to secure data transmitted across networks and ...

...to secure data stored on computers/digital storage



Classification:



Today's Task

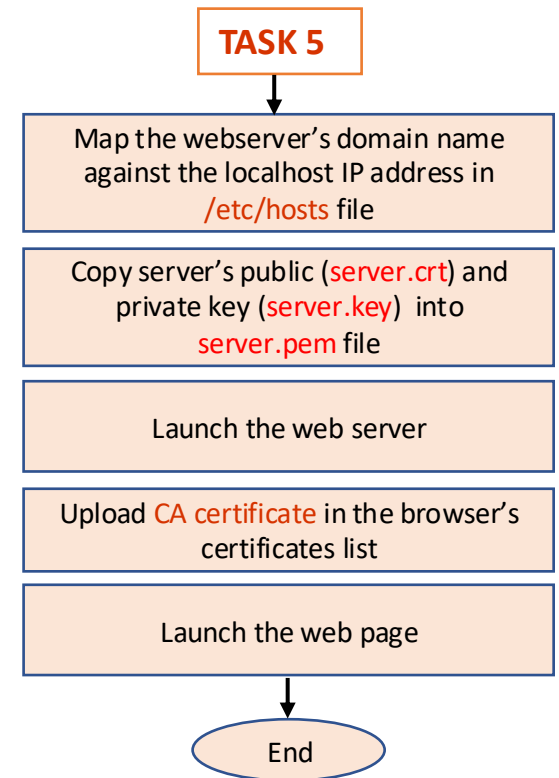
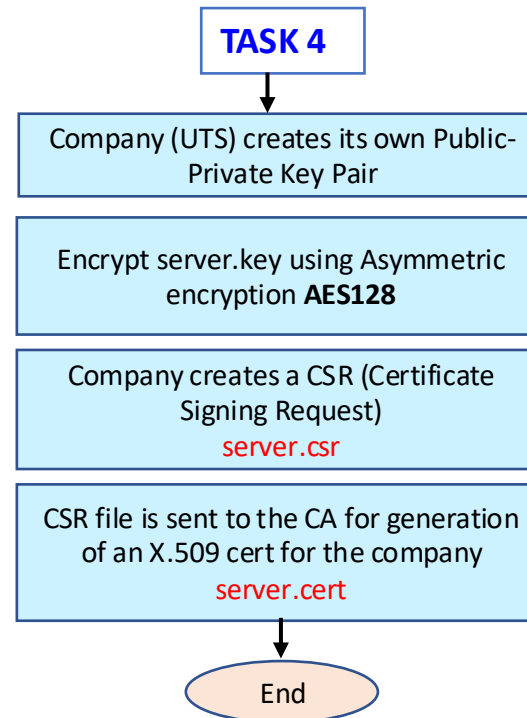
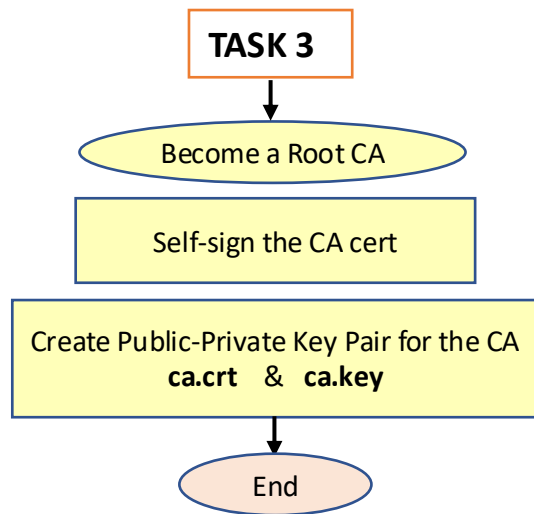
Task 1: Basic encryption and decryption using OpenSSL

Task 2: Asymmetric Encryption

Task 3: Become a Certificate Authority (CA)

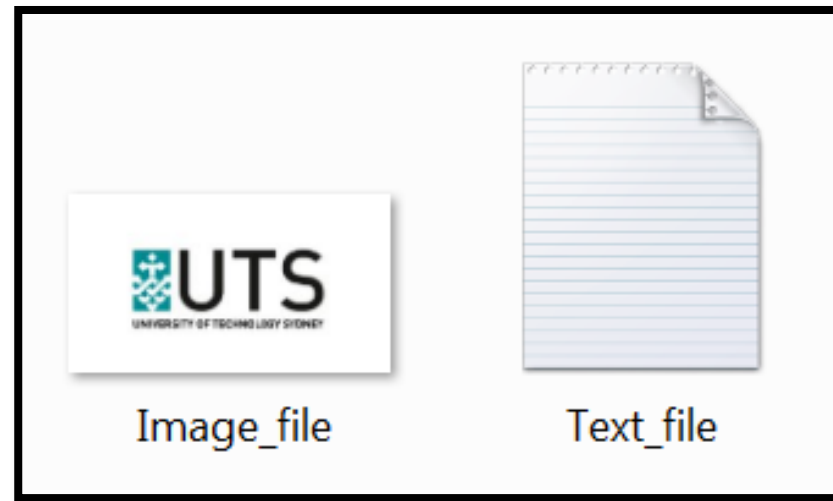
Task 4: Create a Certificate for cybersec.com.au

Task 5: Use PKI for Web Sites



Task 1 Basic Encryption and Decryption

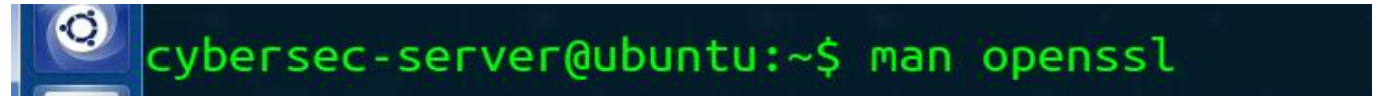
- Download the [plain text file](#), and an [image file](#) from UTS canvas and copy it to the Server [VM \(desktop of VM\)](#)



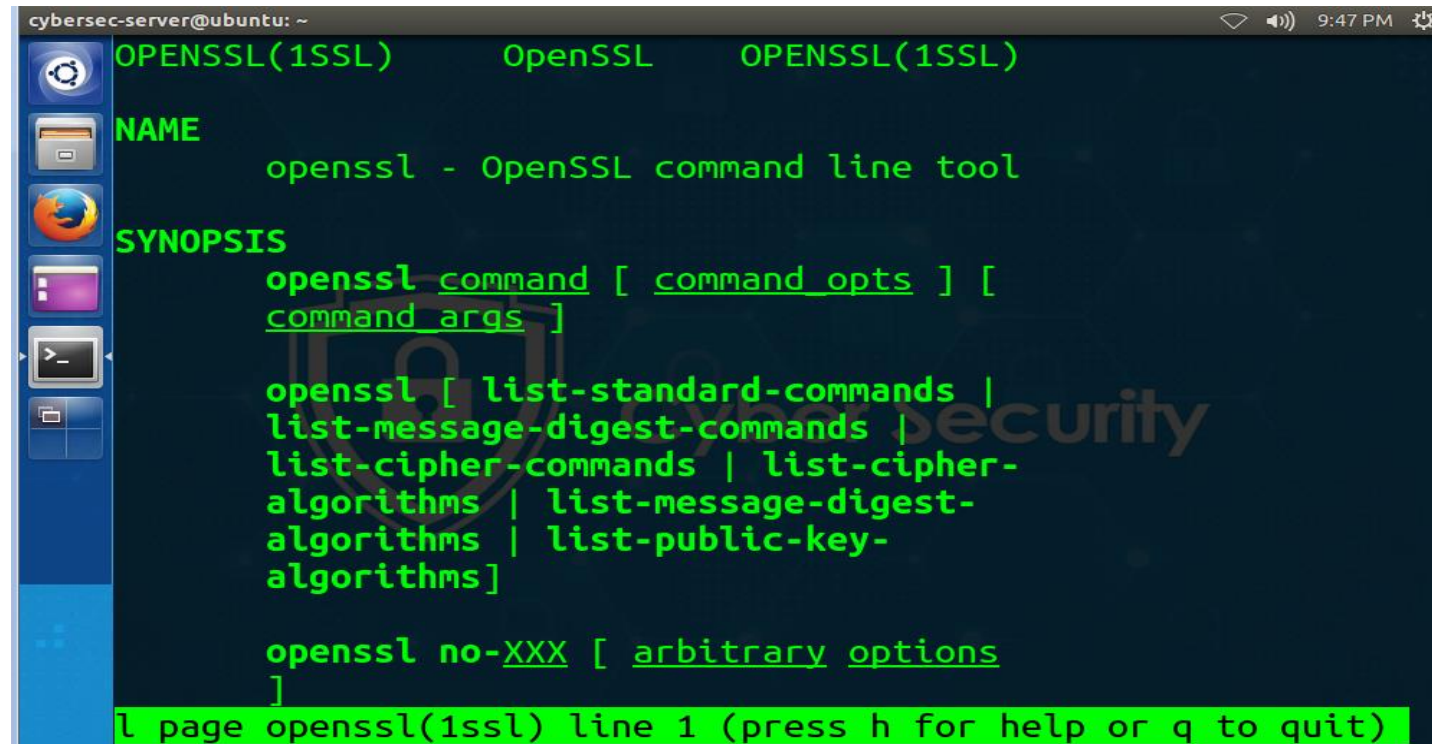
Task 1 Basic Encryption and Decryption

- Use various encryption algorithms and modes on the **Text File** and **Image File** provided
- Save each type of encrypted file separately and **observe** if each encryption technique gives the same or different results
- Please take **screenshots** to show your observations
- The task is to be performed using a **Terminal** (Can be opened from the left task bar or can be run using Ctrl+Alt+T)

- The syntax and format of using **openssl** command



```
cybersec-server@ubuntu:~$ man openssl
```



```
cybersec-server@ubuntu: ~
OPENSSL(1SSL)      OpenSSL      OPENSSL(1SSL)

NAME
    openssl - OpenSSL command line tool

SYNOPSIS
    openssl command [ command_opts ] [
    command_args ]

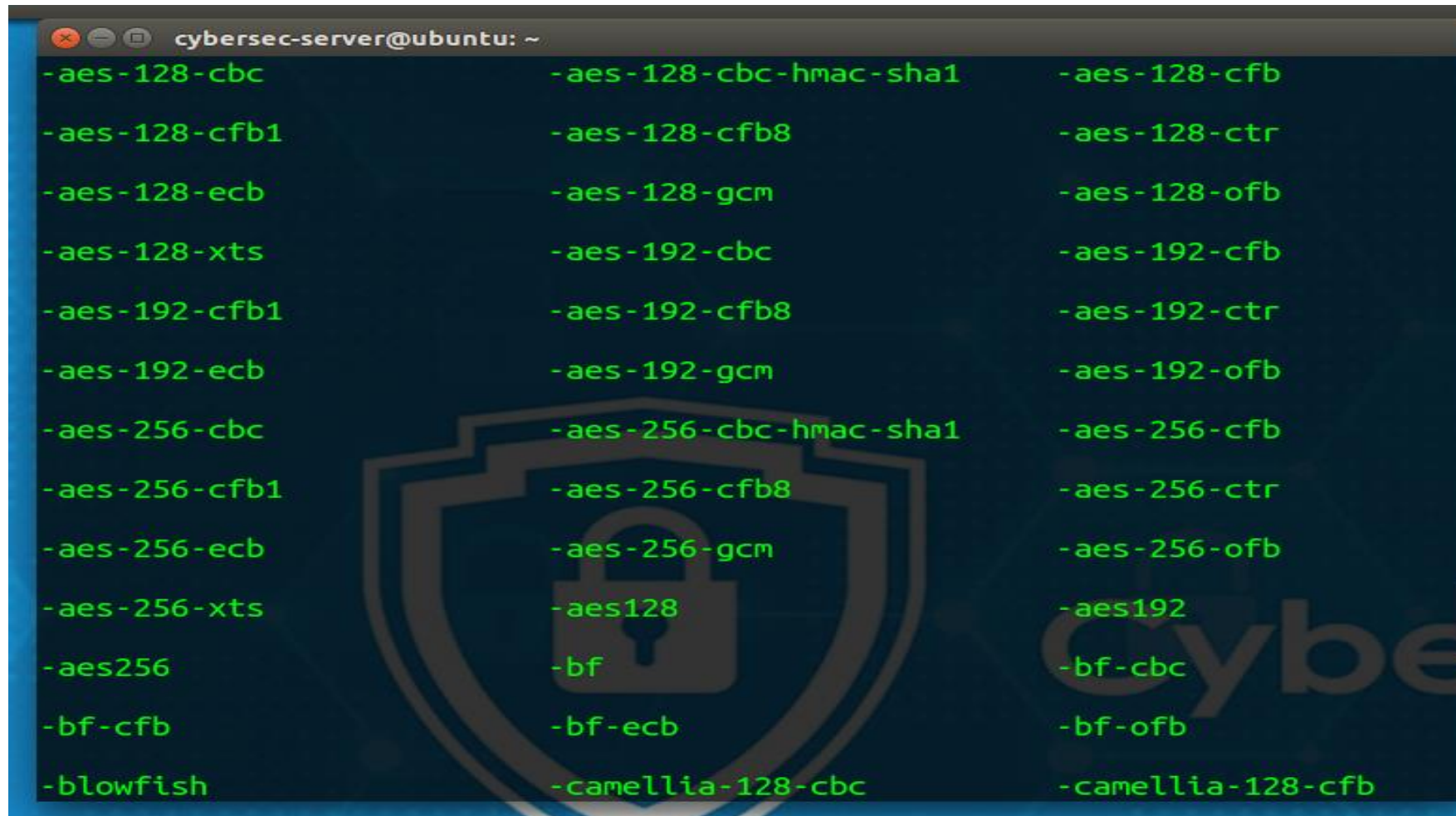
    openssl [ list-standard-commands |
    list-message-digest-commands |
    list-cipher-commands | list-cipher-
    algorithms | list-message-digest-
    algorithms | list-public-key-
    algorithms ]

    openssl no-XXX [ arbitrary options
    ]

l page openssl(1ssl) line 1 (press h for help or q to quit)
```

Task 1 Basic Encryption and Decryption

- Run `openssl enc --help` to list all the ciphers

A terminal window titled 'cybersec-server@ubuntu: ~' displays the output of the 'openssl enc --help' command. The output is a list of 30 encryption and decryption modes, organized into three columns. The modes include AES in various key sizes (128, 192, 256 bits) and modes (cbc, cfb, ecb, ctr, gcm, hmac-sha1, ofb, xts), as well as Blowfish, Camellia, and BF modes. A large, semi-transparent watermark of a shield with a padlock and the word 'Cyber' is visible in the background of the terminal window.

```
cybersec-server@ubuntu: ~  
-aes-128-cbc -aes-128-cbc-hmac-sha1 -aes-128-cfb  
-aes-128-cfb1 -aes-128-cfb8 -aes-128-ctr  
-aes-128-ecb -aes-128-gcm -aes-128-ofb  
-aes-128-xts -aes-192-cbc -aes-192-cfb  
-aes-192-cfb1 -aes-192-cfb8 -aes-192-ctr  
-aes-192-ecb -aes-192-gcm -aes-192-ofb  
-aes-256-cbc -aes-256-cbc-hmac-sha1 -aes-256-cfb  
-aes-256-cfb1 -aes-256-cfb8 -aes-256-ctr  
-aes-256-ecb -aes-256-gcm -aes-256-ofb  
-aes-256-xts -aes128 -aes192  
-aes256 -bf -bf-cbc  
-bf-cfb -bf-ecb -bf-ofb  
-blowfish -camellia-128-cbc -camellia-128-cfb
```

```

cybersec-server@ubuntu:~$ openssl enc --help
unknown option '--help'
options are
-in <file>          input file
-out <file>          output file
-pass <arg>          pass phrase source
-e                  encrypt
-d                  decrypt
-a/-base64          base64 encode/decode, depending on encryption flag
-k                  passphrase is the next argument
-kfile              passphrase is the first line of the file argument
-md                the next argument is the md to use to create a key
                    from a passphrase. One of md2, md5, sha or sha1
-S                  salt in hex is the next argument
-K/-iv              key/iv in hex is the next argument
-[pP]              print the iv/key (then exit if -P)
-bufsize <n>        buffer size
-nopad              disable standard block padding
-engine e           use engine e, possibly a hardware device.

Cipher Types
-aes-128-cbc          -aes-128-cbc-hmac-sha1      -aes-128-cfb
-aes-128-cfb1         -aes-128-cfb8                -aes-128-ctr
-aes-128-ecb          -aes-128-gcm                  -aes-128-ofb
-aes-128-xts          -aes-192-cbc                -aes-192-cfb
-aes-192-cfb1         -aes-192-cfb8                -aes-192-ctr
-aes-192-ecb          -aes-192-gcm                  -aes-192-ofb
-aes-256-cbc          -aes-256-cbc-hmac-sha1      -aes-256-cfb
-aes-256-cfb1         -aes-256-cfb8                -aes-256-ctr
-aes-256-ecb          -aes-256-gcm                  -aes-256-ofb
-aes-256-xts          -aes128                      -aes192
-aes256               -bf                          -bf-cbc
-bf-cfb               -bf-ecb                      -bf-ofb
-blowfish             -camellia-128-cbc          -camellia-128-cfb
-camellia-128-cfb1    -camellia-128-cfb8          -camellia-128-ecb
-camellia-128-ofb     -camellia-192-cbc          -camellia-192-cfb
-camellia-192-cfb1    -camellia-192-cfb8          -camellia-192-ecb
-camellia-192-ofb     -camellia-256-cbc          -camellia-256-cfb
-camellia-256-cfb1    -camellia-256-cfb8          -camellia-256-ecb
-camellia-256-ofb     -camellia128                -camellia192
-camellia256          -cast                        -cast-cbc
-cast5-cbc            -cast5-cfb                  -cast5-ecb
-cast5-ofb            -des                         -des-cbc
-des-cfb              -des-cfb1                   -des-cfb8
-des-ecb              -des-edc                     -des-edc-cbc
-des-edc-cfb          -des-edc-ofb                 -des-edc3
-des-edc3-cbc         -des-edc3-cfb                -des-edc3-cfb1
-des-edc3-cfb8        -des-edc3-ofb                -des-ofb
-des3                 -desx                        -desx-cbc
-id-aes128-GCM         -id-aes192-GCM               -id-aes256-GCM
-rc2                   -rc2-40-cbc                  -rc2-64-cbc
-rc2-cbc              -rc2-cfb                     -rc2-ecb
-rc2-ofb               -rc4                          -rc4-40
-rc4-hmac-md5          -seed                         -seed-cbc
-seed-cfb              -seed-ecb                     -seed-ofb

```

```
cybersec-server@ubuntu:~$ cd Desktop
cybersec-server@ubuntu:~/Desktop$ ls
image.bmp  Text_file.txt
cybersec-server@ubuntu:~/Desktop$ openssl enc -aes-128-cbc -e -in Text_File.txt -out Encrypted.enc -K 0011223344556677889900 -iv 0000
Text_File.txt: No such file or directory
139980804744864:error:02001002:system library:fopen:No such file or directory:bss_file.c:398:fopen('Text_File.txt','r')
139980804744864:error:20074002:BIIO routines:FILE_CTRL:system lib:bss_file.c:400:
cybersec-server@ubuntu:~/Desktop$ openssl enc -aes-128-cbc -e -in Text file.txt -out Encrypted.enc -K 0011223344556677889900 -iv 0000
cybersec-server@ubuntu:~/Desktop$
```

Task 1

- -Next, navigate to the desktop and then we will be “encrypting the text file” using “-aes-128-cbc” cipher type since the text file is present in the “Desktop”
- Encryption
- a.We do encryption and decryption here using “-aes-128-cbc”
- “openssl enc -aes-128-cbc -e -in Text_File.txt -out Encrypted.enc -K 0011223344556677889900 -iv 0000”
- NOTE:- The key has to be the same for encryption and decryption

Task 1 Basic Encryption and Decryption

`openssl enc <cipher-type> -e/-d -in <> -out <> -K <> -iv <>`

- `enc` command to encrypt
- `Cipher-type` CBC, CFB, ECB, OFB
- `-e` for encrypt
- `-d` for decrypt
- `-in <input file>` Plain-text file to be encrypted
- `-out <output file>` File in which encrypted text will be written
- `-K <secret key>` Key to be used for **encryption**
(Same key to be used for **decryption**)
- `-iv <initialization vector>` This is required for block cipher

Encryption

`openssl enc -aes-128-cbc -e -in Text_file.txt -out Encrypted.enc -K 11223344556677889900 -iv 0000`

Decryption

`openssl enc -aes-128-cbc -d -in Encrypted.enc -out Decrypted.txt -K 11223344556677889900 -iv 0000`

-We can view
the encrypted
file using “cat
Encrypted.enc”

```
cybersec-server@ubuntu:~/Desktop$ cat Encrypted.enc
D  A;bZ$G?o7{c$SL_ .{G ]X{ }_  IAf fS8FomS 2p6RDuon}SR(2_{?A
*/Ko, R- 3Toy=1^lh p_ 2 J*SWEdufPxxvU?#x:x b
w]d
r[^+ex{hB}LAC@B; c i^} 1 199T(PGPpr`'
#g_8G|7D| / 8RHa{tC}7_b8 q: A@
  r30Vso83vGokTCDVg DkCZzhsdewePod_
+<_yope*
$ 0 N=3 A  Y#mEz)Dlwe l t]P a\B#Qoj` D B2}xTDF H; $
lyfXzIeQ7f%{E#EeNh1~ bL3
  vPZeAiSej,4
MrB(94E
"J
<_@ aIY:Wnnumey 7hne'~+ %  <wxW
(- =\MmE[Xye{ 0 =(1j 7 < >  N
      =G-n      8p0 -kF$zXgk -t xK@Q b\ 'a+ \g4 uqqE8m^ =1vS=
untu:~/Desktop$
```

Decryption:-

-We write the similar command

```
“openssl enc -aes-128-cbc -d -in  
Encrypted.enc -out Decrypted.txt -K  
0011223344556677889900 -iv 0000”
```

```
untu:~/Desktop$ openssl enc -aes-128-cbc -d -in Encrypted.enc -out Decrypted.txt -K 0011223344556677889900 -iv 0000
```

```
cybersec-server@ubuntu:~/Desktop$ cat Decrypted.txt
```

Cryptolocker is a class of malware known as ransomware, which infects a computer by restricting access to that computer's files. Once a computer is infected, the luckless user receives a message demanding payment before files can be unlocked again. Payment is typically demanded in digital currency, such as BitCoins, which is worldwide, untraceable, and can be converted back into cash at the receiving end.

As ransomware goes, Cryptolocker is particularly nasty. A user activates it by opening an email attachment that contains the virus, which then installs and spreads onto the computer, encrypting files as it goes. In the case of a network, an infected machine serves as the gateway to the entire network, encrypting all files the infected user has access to and rendering these files inaccessible. In order to recover encrypted files the victim is asked to pay a ransom, typically within 72 hours of infection, or lose all data forever. In an astounding new twist to this story, the cybercriminals who created Cryptolocker have upped the ante by creating a "customer service" site designed to help victims who are unable to make the initial ransom payment. Through this site, victims are given an extended deadline to come up with the ransom payment, but in this case the ransom is considerably higher than the initial demand.

Task 2 – Asymmetric Encryption

- **Commands**

1. `echo 'rishi'>file.txt`

2. `cat file.txt`

Step 1: Generating RSA Key Pair

1. `openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048`

2. `cat private.pem`

3. `openssl rsa -pubout -in private.pem -out public.pem`

4. `cat public.pem`

Asymmetric encryption

Step 2: Encryption

1. openssl rsautl -encrypt -inkey public.pem -pubin -in file.txt -out file.enc

2. cat file.enc

Step 3: Decryption

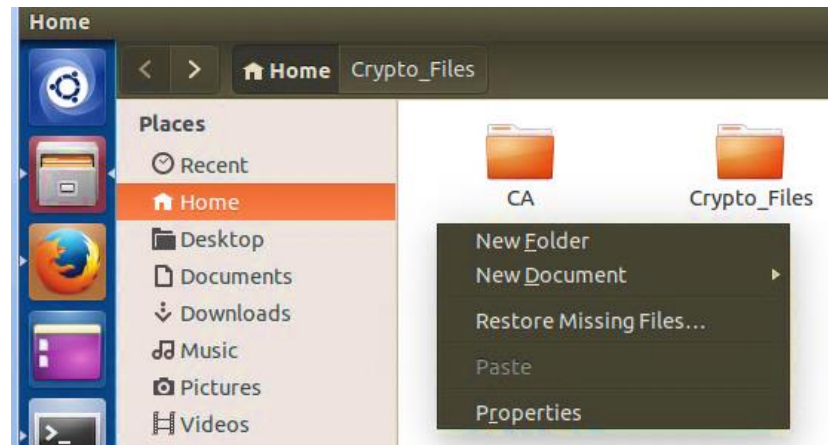
1. openssl rsautl -decrypt -inkey private.pem -in file.enc -out dec.txt

2. cat dec.txt

Task 3: Become a Certificate Authority

Step 1: The Configuration files

- To start with, we need to first create a copy of the **openssl.cnf** file and supporting directories
- To do this, create a new folder called **CA** in the Home directory

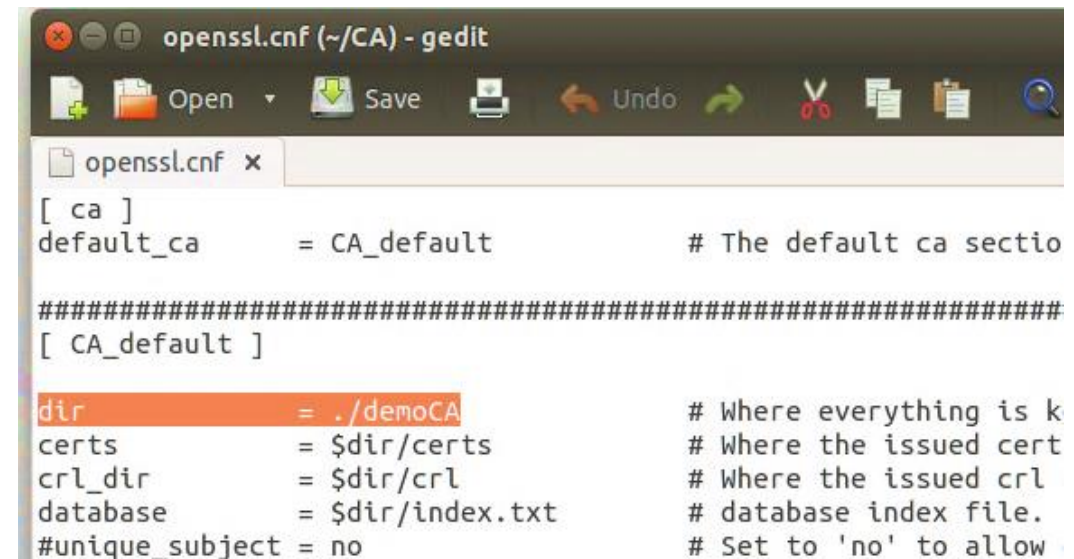


Go to **CA** directory

Now for the **openssl.cnf** file, use the following to copy it to the **current directory**.

```
cp /usr/lib/ssl/openssl.cnf ./
```

- Then we need to make a change in the openssl.cnf (use gedit) file as shown below:



- **Remove** “demoCA”, and then **save** and close the file

Task 3: Become a Certificate Authority

- First create a directory “CA”

```
cybersec-server@ubuntu:~$ mkdir CA
cybersec-server@ubuntu:~$ ls
CA      Documents  examples.desktop  Pictures  Templates
Desktop Downloads  Music             Public    Videos
cybersec-server@ubuntu:~$
```



- -Now, we copy the openssl.cnf file to the desktop using
- “cp /usr/lib/ssl/openssl.cnf ./”

```
cybersec-server@ubuntu: ~/CA
cybersec-server@ubuntu:~$ mkdir CA
cybersec-server@ubuntu:~$ ls
CA      Documents  examples.desktop  Pictures  Templates
Desktop Downloads  Music             Public    Videos
cybersec-server@ubuntu:~$ cd CA
cybersec-server@ubuntu:~/CA$ cp /usr/lib/ssl/openssl.cnf ./
cybersec-server@ubuntu:~/CA$
```

```

*openssl.cnf x
#
# OpenSSL example configuration file.
# This is mostly being used for generation of certificate requests.
#
# This definition stops the following lines choking if HOME isn't
# defined.
HOME                = .
RANDFILE            = $ENV::HOME/.rnd

# Extra OBJECT IDENTIFIER info:
#oid_file            = $ENV::HOME/.oid
oid_section          = new_oids

# To use this configuration file with the "-extfile" option of the
# "openssl x509" utility, name here the section containing the
# X.509v3 extensions to use:
# extensions         =
# (Alternatively, use a configuration file that has only
# X.509v3 extensions in its main [= default] section.)

[ new_oids ]

# We can add new OIDs in here for use by 'ca', 'req' and 'ts'.
# Add a simple OID like this:
# testoid1=1.2.3.4
# Or use config file substitution like this:
# testoid2=${testoid1}.5.6

# Policies used by the TSA examples.
tsa_policy1 = 1.2.3.4.1
tsa_policy2 = 1.2.3.4.5.6
tsa_policy3 = 1.2.3.4.5.7

#####
[ ca ]
default_ca      = CA_default      # The default ca section

#####
[ CA_default ]

dir              = ./              # Where everything is kept
certs            = $dir/certs      # Where the issued certs are kept
crl_dir          = $dir/crl        # Where the issued crl are kept
database         = $dir/index.txt  # database index file.

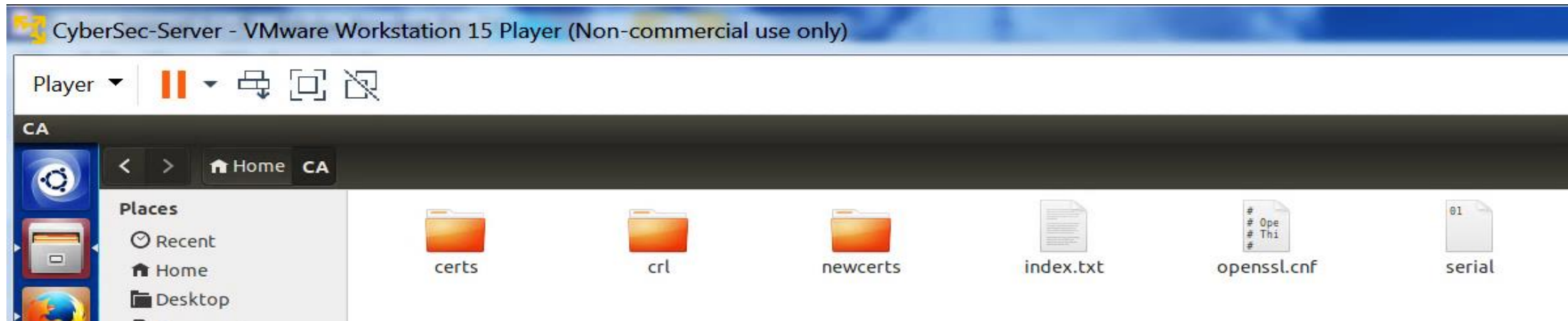
```

We change the dir
from ./demoCA to ./

- -We make a change to openssl.conf file by changing
- dir = ./demoCA -> dir = ./

Task 3: Become a Certificate Authority

- Get the additional folders/files in the **CA** folder by using the following commands:
 - `mkdir certs crl newcerts` #Making 3 new directories
 - `touch index.txt` #Creating a new file index.txt
 - `echo '01' > serial` #Creating a new file serial with '01' written in it



Task 3: Become a Certificate Authority

Step 2: Certificate Authority

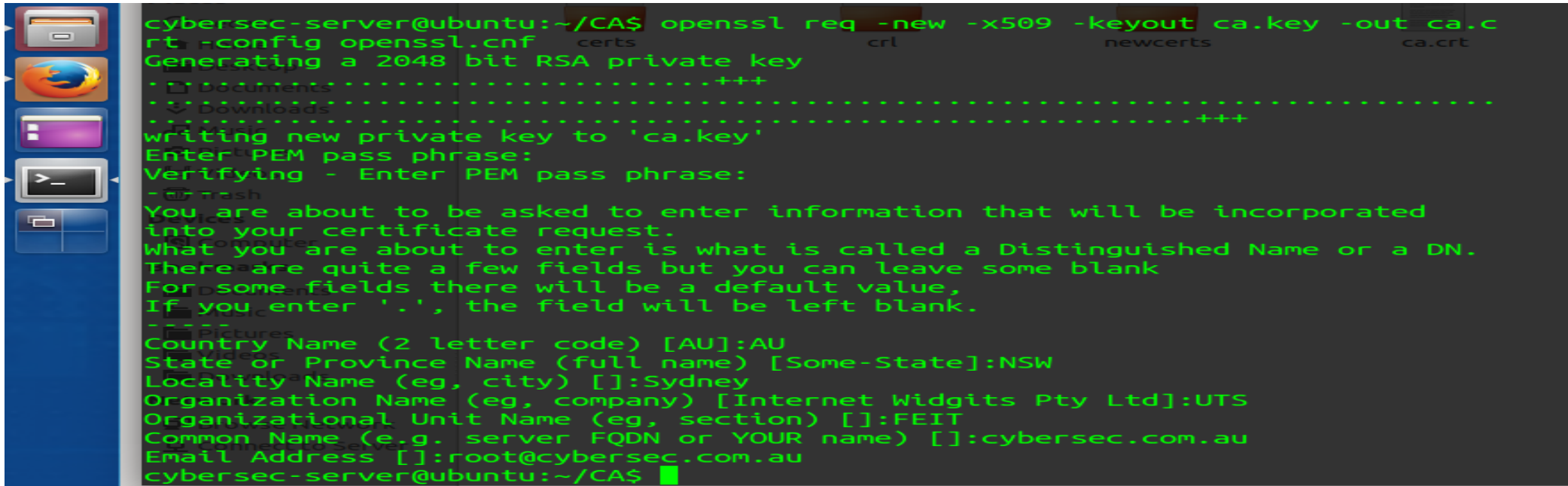
- Once the files and folders are made, next, we need to generate a self-signed certificate for ourselves, the **root CA**
- This means that this CA is totally trusted, and its certificate will serve as the root certificate.

➤ Use this command to create/issue a new **X.509** certificate to cybersec.com.au
`openssl req -new -x509 -keyout ca.key -out ca.crt -config openssl.cnf`

Note: Run this command in the terminal from CA directory

- Use [cybersec](https://cybersec.com.au) as the passphrase everywhere
- You will be asked to enter some information
 - Country Name (2 letter code) [AU]: **AU**
 - State or Province Name (full name) [Some-State]: **NSW**
 - Locality Name (eg, city) []: **Sydney**
 - Organization Name (eg, company) [Internet Widgits Pty Ltd]: **UTS**
 - Organizational Unit Name (eg, section) []: **FEIT**
 - Common Name (e.g. server FQDN or YOUR name) []: **cybersec.com.au**
 - Email Address []: **root@cybersec.com.au**

Task 3: Become a Certificate Authority



```
cybersec-server@ubuntu:~/CA$ openssl req -new -x509 -keyout ca.key -out ca.crt -config openssl.cnf
Generating a 2048 bit RSA private key
writing new private key to 'ca.key'
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
Country Name (2 letter code) [AU]:AU
State or Province Name (full name) [Some-State]:NSW
Locality Name (eg, city) []:Sydney
Organization Name (eg, company) [Internet Widgits Pty Ltd]:UTS
Organizational Unit Name (eg, section) []:FEIT
Common Name (e.g. server FQDN or YOUR name) []:cybersec.com.au
Email Address []:root@cybersec.com.au
cybersec-server@ubuntu:~/CA$
```

- The output of the command is stored in two files: **ca.key** and **ca.crt**. The file **ca.key** contains the CA's private key, while **ca.crt** contains the public-key certificate. Now our private key and the certificate is ready as per the screenshot mentioned



Task 4: Create a Certificate for cybersec.com.au

- Now, we become a root CA, we are ready to sign digital certificates for our customers
- Our first customer is a company called **cybersec.com.au**. For this company to get a digital certificate from a CA, it needs to go through **three steps**

Task 4: Create a Certificate for cybersec.com.au

Step 1: Generate public/private key pair

- The company needs to first **create its own public/private key pair**
- We can run the following command to generate an **RSA key pair** (both private and public keys) again from **CA** directory
- You are also required to **provide a password to encrypt the private key** (using the AES-128 encryption algorithm)
- The keys will be stored in the file **server.key**:

```
openssl genrsa -aes128 -out server.key 1024
```

Task 4: Create a Certificate for cybersec.com.au

```
cybersec-server@ubuntu:~/CA$ openssl genrsa -aes128 -out server.key 1024
Generating RSA private key, 1024 bit long modulus
.....+++++
Network.....+++++
e is 65537 (0x10001)
Enter pass phrase for server.key:
Verifying - Enter pass phrase for server.key:
cybersec-server@ubuntu:~/CA$
```

CA newcerts



certs



crl



newcerts



ca.crt



ca.key



index.txt



openssl.cnf



serial



server.key

Task 4: Create a Certificate for cybersec.com.au

Step 2: Generate a Certificate Signing Request (CSR)

- Once the company has the key file, it should generate a **Certificate Signing Request (CSR)**, which basically includes the company's public key
- The CSR will be sent to the CA, who will generate a certificate for the key (usually after ensuring that identity information in the CSR matches with the server's identity)
- Use **cybersec.com.au** as the common name of the certificate request

```
openssl req -new -key server.key -out server.csr -config  
openssl.cnf
```

You will be once again prompted for information and a password, use the following details and keep in mind they are case-sensitive and should be same as the ones used before:

- Country Name (2 letter code) [AU]: **AU**
- State or Province Name (full name) [Some-State]: **NSW**
- Locality Name (eg, city) []: **Sydney**
- Organization Name (eg, company) [Internet Widgits Pty Ltd]: **UTS**
- Organizational Unit Name (eg, section) []: **FEIT**
- Common Name (e.g. server FQDN or YOUR name) []: **cybersec.com.au**
- Email Address []: **root@cybersec.com.au**
- Please enter the following 'extra' attributes to be sent with your certificate request
 - A challenge password []: < **LEAVE BLANK** >
 - An optional company name []: < **LEAVE BLANK** >

Task 4: Create a Certificate for cybersec.com.au

```
cybersec-server@ubuntu:~/CA$ openssl req -new -key server.key -out server.csr -config openssl.cnf
Enter pass phrase for server.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:AU
State or Province Name (full name) [Some-State]:NSW
Locality Name (eg, city) []:Sydney
Organization Name (eg, company) [Internet Widgits Pty Ltd]:UTS
Organizational Unit Name (eg, section) []:FEIT
Common Name (e.g. server FQDN or YOUR name) []:cybersec.com.au
Email Address []:root@cybersec.com.au

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
cybersec-server@ubuntu:~/CA$
```



Task 4: Create a Certificate for cybersec.com.au

Step 3: Generating Certificates

- The CSR file needs to have the CA's signature to form a certificate
- In the real world, the CSR files are usually sent to a trusted CA for their signature
- In this lab, we will use our own trusted CA to generate certificates
- The following command turns the certificate signing request (**server.csr**) into an X.509 certificate (**server.crt**), using the CA's **ca.crt** and **ca.key**:

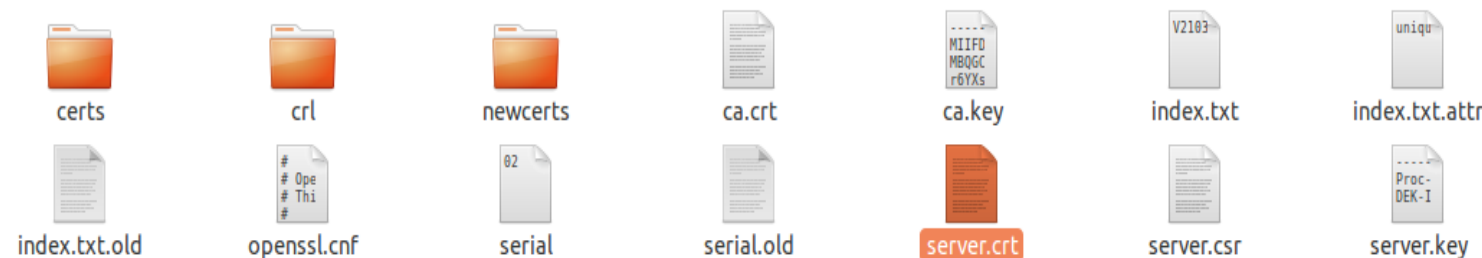
```
openssl ca -in server.csr -out server.crt -cert ca.crt -keyfile ca.key -  
config openssl.cnf
```

Task 4: Create a Certificate for cybersec.com.au

```
Terminal
cybersec-server@ubuntu: ~/CA
An optional company name []:
cybersec-server@ubuntu:~/CA$ openssl ca -in server.csr -out server.crt -cert ca.crt -keyfile ca.key -config openssl.cnf
Using configuration from openssl.cnf
Enter pass phrase for ca.key:
Check that the request matches the signature
Signature ok
Certificate Details:
  Serial Number: 1 (0x1)
  Validity
    Not Before: Mar 22 09:23:00 2020 GMT
    Not After : Mar 22 09:23:00 2021 GMT
  Subject:
    countryName           = AU
    stateOrProvinceName   = NSW
    organizationName      = UTS
    organizationalUnitName = FEIT
    commonName            = cybersec.com.au
    emailAddress          = root@cybersec.com.au
  X509v3 extensions:
    X509v3 Basic Constraints:
      CA:FALSE
    Netscape Comment:
      OpenSSL Generated Certificate
    X509v3 Subject Key Identifier:
      D6:1B:CC:DD:36:6E:1F:E6:D1:20:1A:43:11:B7:AF:66:50:40:28:10
    X509v3 Authority Key Identifier:
      keyid:50:9E:0C:B2:6F:A8:4F:DF:2D:E4:1C:77:96:A7:91:89:2F:EF:7F:B2

Certificate is to be certified until Mar 22 09:23:00 2021 GMT (365 days)
Sign the certificate? [y/n]:y

1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
cybersec-server@ubuntu:~/CA$
```



Task 5: Use PKI for Web Sites

- In this lab, we will explore how public-key certificates are used by web sites to secure web browsing
- First, we need to get our domain name
- Let us use **cybersec.com.au** as our domain name

Step 1

- To get our computers recognize this domain name, let us add the following entry to **/etc/hosts**. This entry basically maps the domain name **cybersec.com.au** to our localhost (i.e., 127.0.0.1):

127.0.0.1 cybersec.com.au

Task 5: Use PKI for Web Sites

```
cybersec-server@ubuntu: ~/CA
cybersec-server@ubuntu:~/CA$ sudo gedit /etc/hosts
```

```
hosts x
127.0.0.1 cybersec.com.au
127.0.0.1 netsec.com.au
#127.0.0.1 localhost
#127.0.0.1 sqlinjection.com.au
127.0.1.1 ubuntu

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Add the
cybersec.com.au to the
hosts

Task 5: Use PKI for Web Sites

Step 2

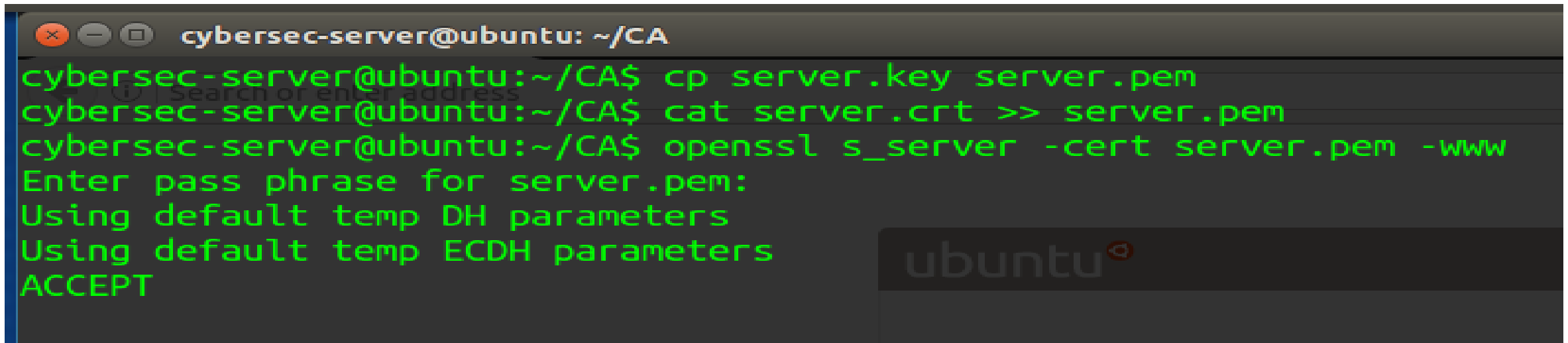
- Next, let us launch a simple web server with the certificate generated in the previous task
- **OpenSSL** allows us to start a simple web server using the **s_server** command:
- Combine the domain's secret key and certificate into one file

```
cp server.key server.pem
```

```
cat server.crt >> server.pem
```

- Launch the web server using server.pem

```
openssl s_server -cert server.pem -www
```

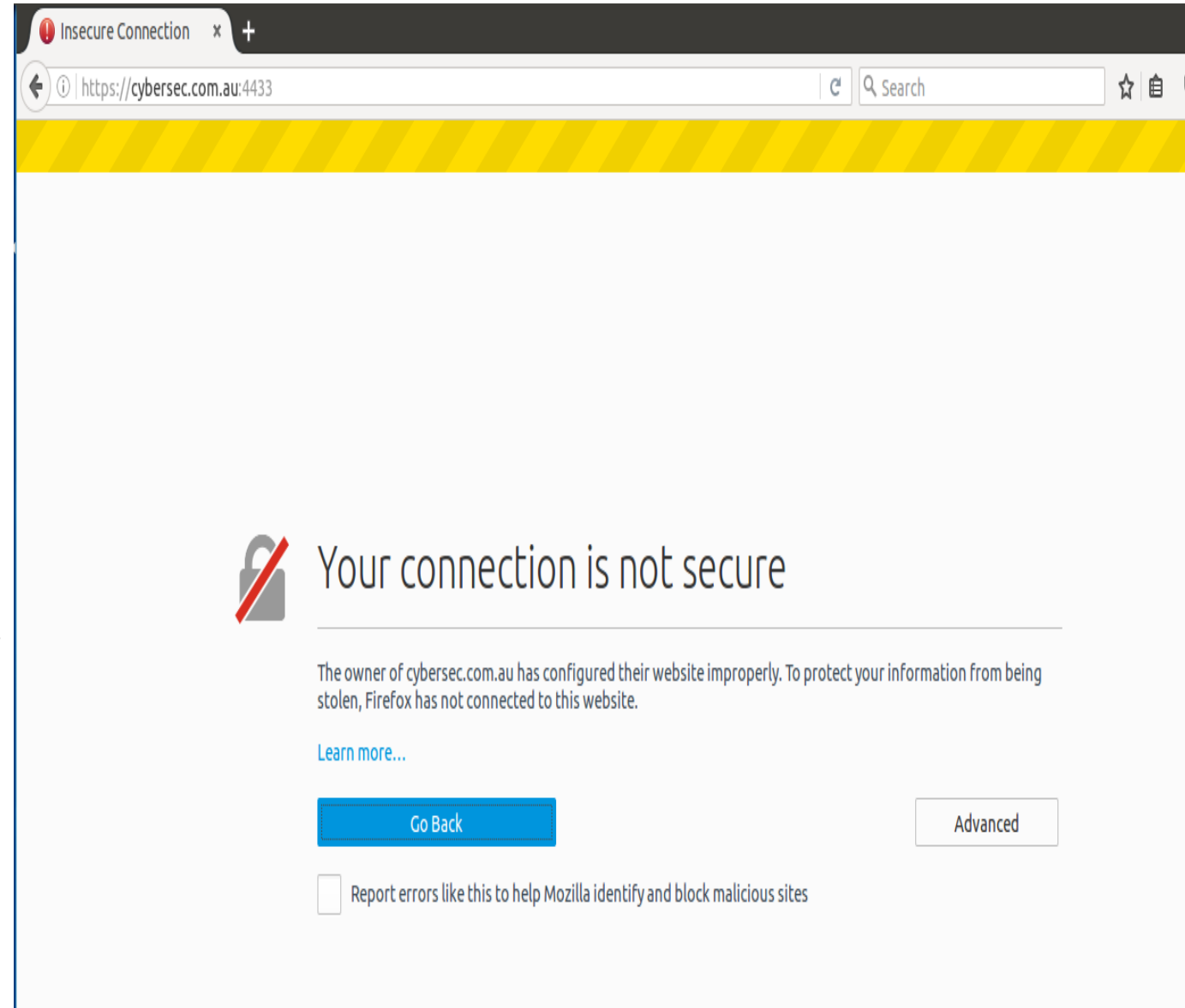


```
cybersec-server@ubuntu: ~/CA
cybersec-server@ubuntu:~/CA$ cp server.key server.pem
cybersec-server@ubuntu:~/CA$ cat server.crt >> server.pem
cybersec-server@ubuntu:~/CA$ openssl s_server -cert server.pem -www
Enter pass phrase for server.pem:
Using default temp DH parameters
Using default temp ECDH parameters
ACCEPT
```

Task 5: Use PKI for Web Sites

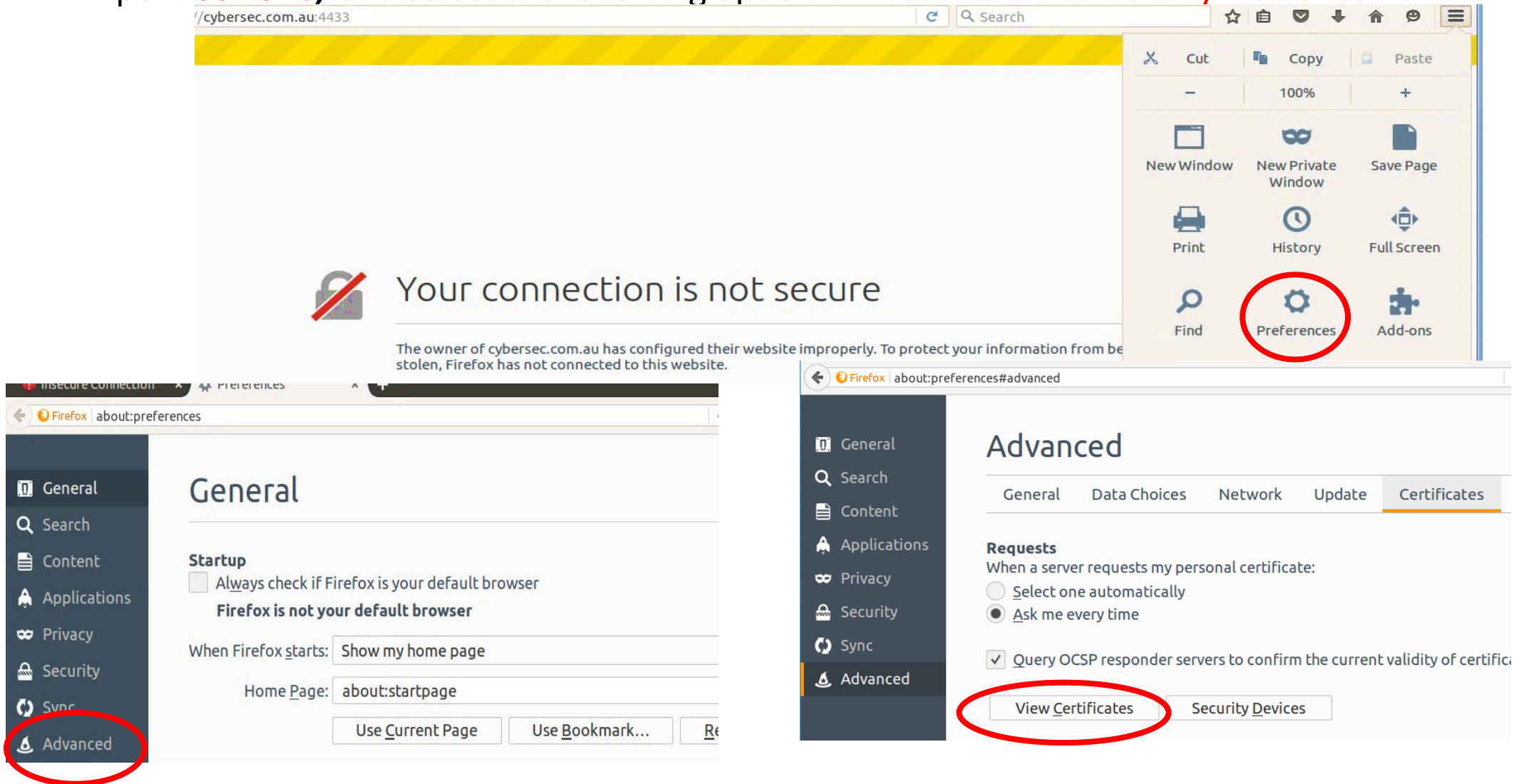
Step 3

- By default, the server will listen on **port 4433**
- You can alter that using the **-accept** option
- Now, you can access the server using the following URL
<https://cybersec.com.au:4433/>
- Had this certificate been assigned by VeriSign, we will not have such an error message.
- Unfortunately, the certificate of cybersec.com.au is signed by our own CA (i.e., using **ca.crt**), and this CA is not recognized by Firefox.
- Therefore, we need to load the ca.cert into Firefox

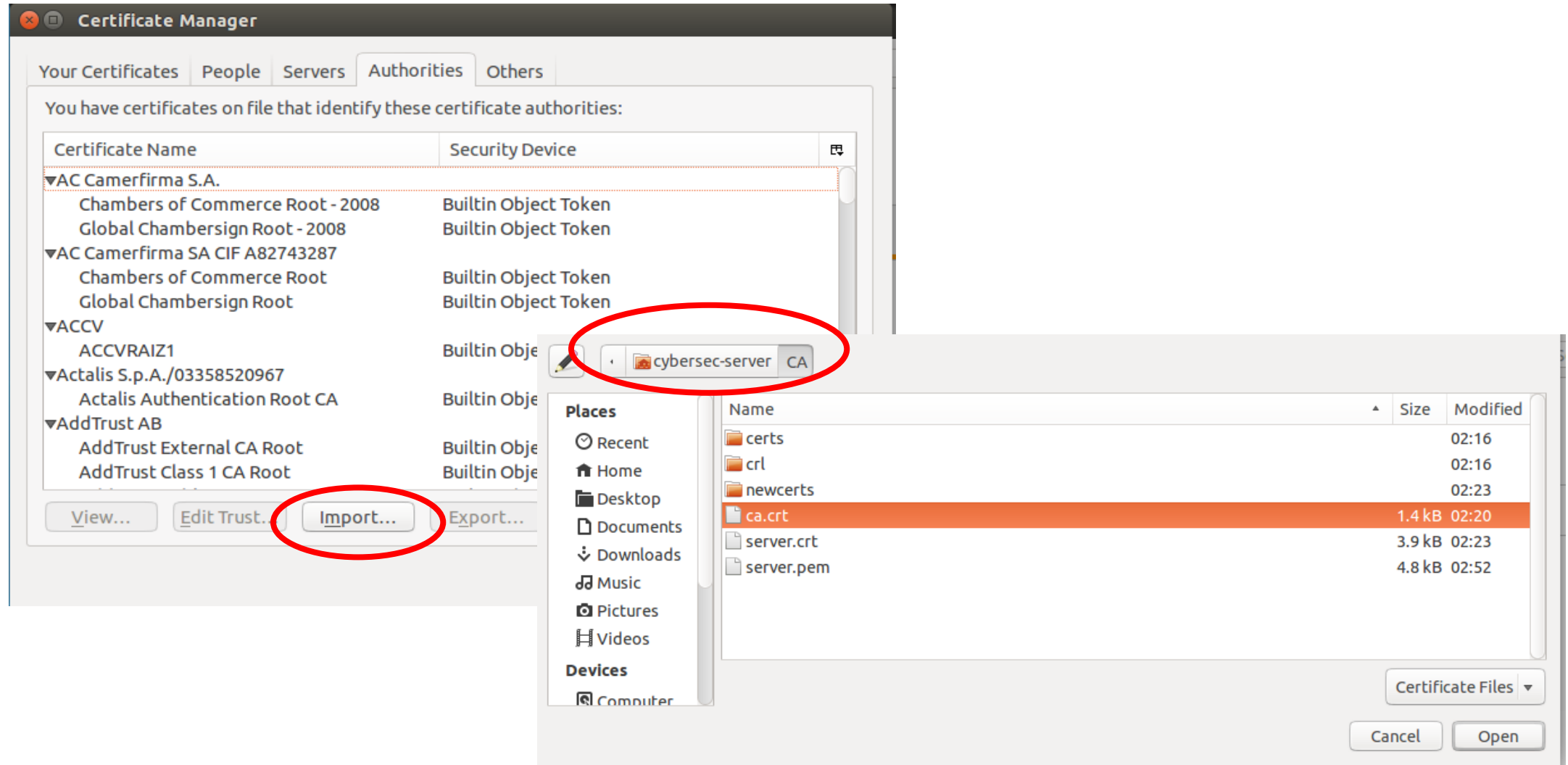


Task 5: Use PKI for Web Sites

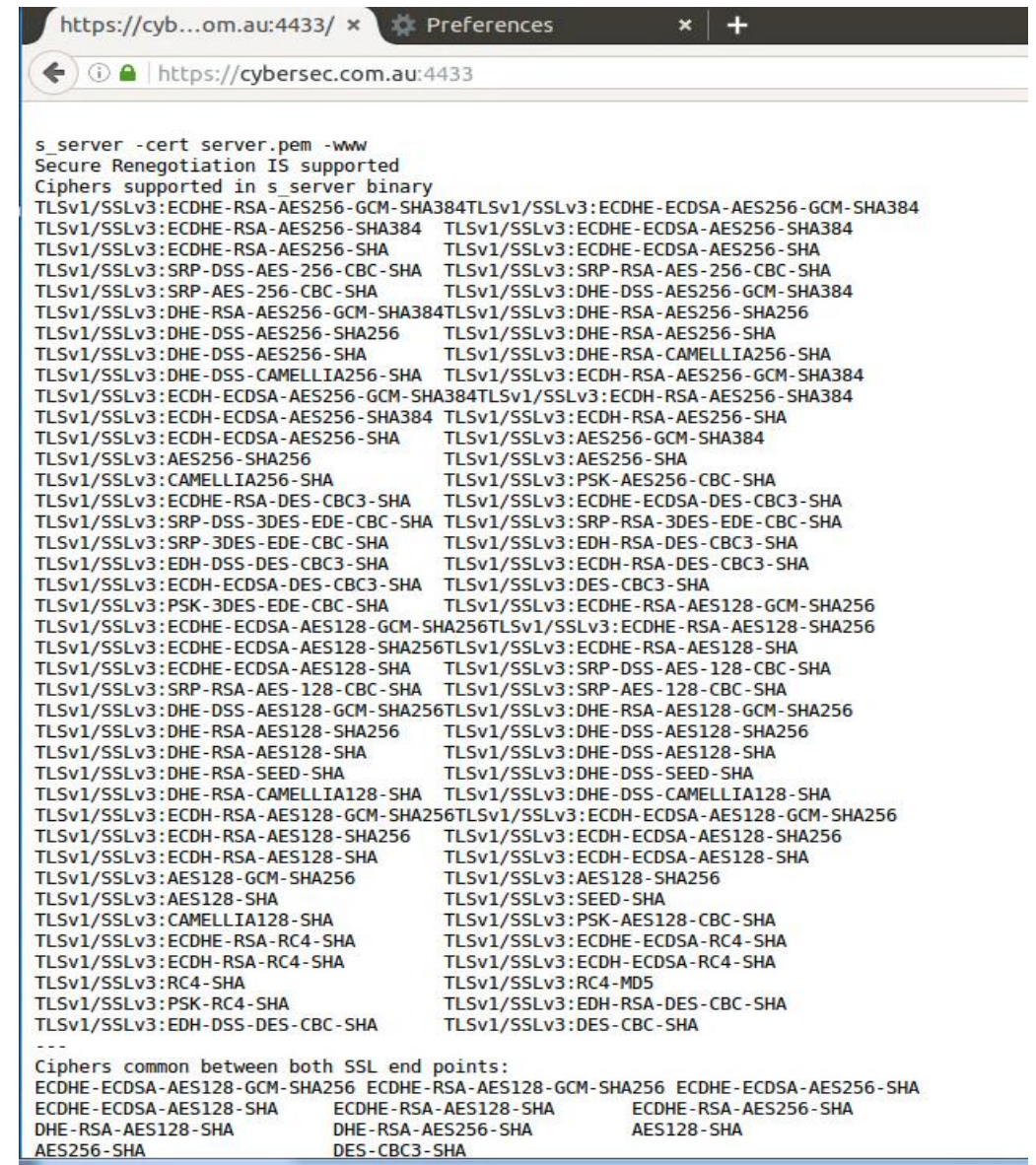
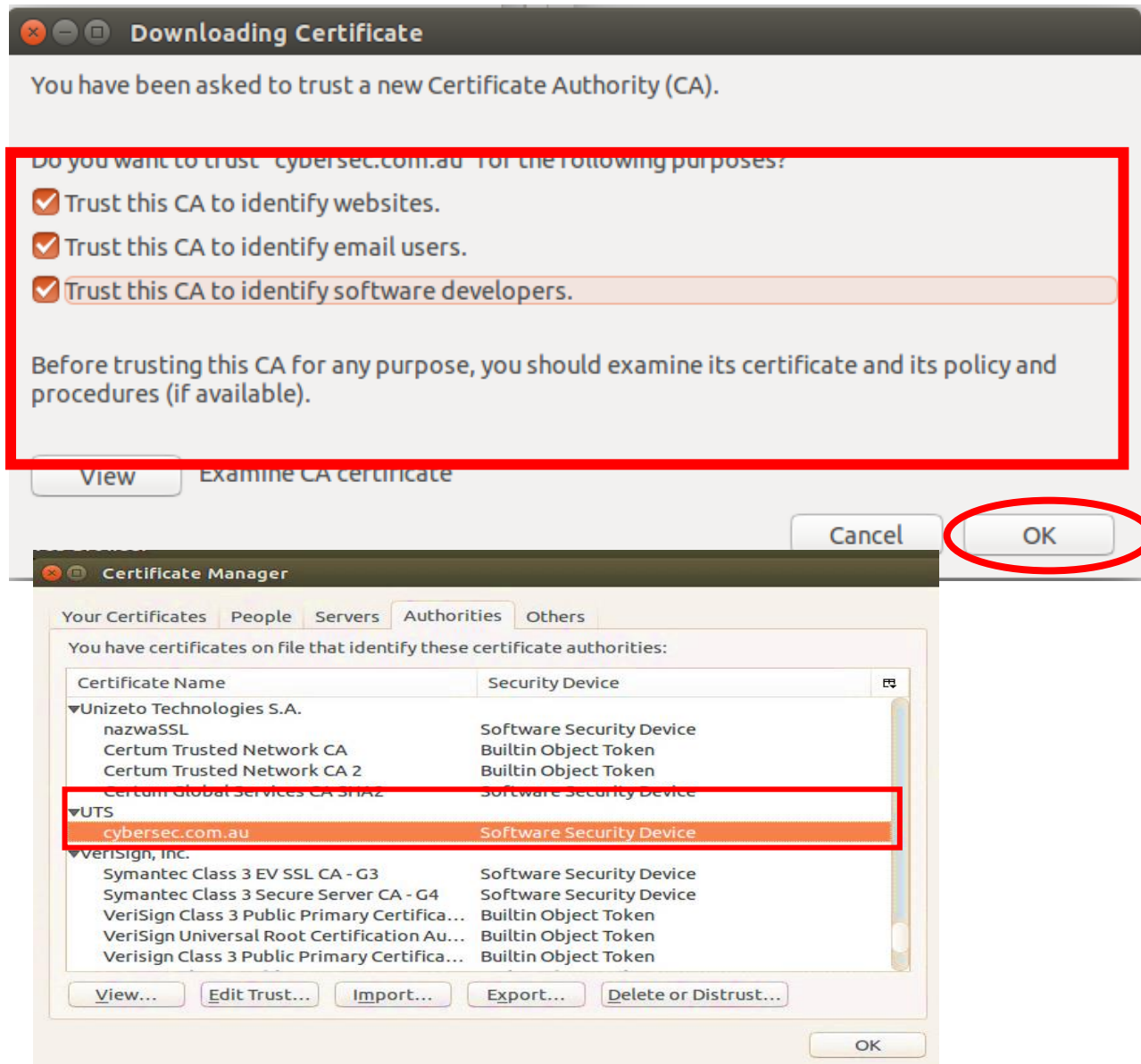
- Import **ca.crt**, and select the following option: “Trust this CA to identify web sites”.



Task 5: Use PKI for Web Sites



Task 5: Use PKI for Web Sites



Summary of Task 3, 4 and 5

TASK 3

Become a Root CA

Self-sign the CA cert

Create Public-Private Key Pair for the CA
ca.crt & **ca.key**

TASK 4

Company (UTS) creates its own Public-Private Key Pair

Encrypt server.key using Asymmetric encryption **AES128**

Company creates a CSR (Certificate Signing Request)
server.csr

CSR file is sent to the CA for generation of an X.509 cert for the company
server.cert

TASK 5

Map the webserver's domain name against the localhost IP address in **/etc/hosts** file

Copy server's public (**server.crt**) and private key (**server.key**) into **server.pem** file

Launch the web server

Upload **CA certificate** in the browser's certificates list

Launch the web server again

End

Tasks to be Performed by the students

- **Task-1:** Basic encryption and decryption using OpenSSL
 - Encryption and decryption of the **Text_file**
 - Encryption and decryption of the **image file**
- **Task-2:** Asymmetric encryption
- **Task-3:** Become a Certificate Authority(CA)
- **Task-4:** Create a Certificate for cybersec.com.au
- **Task 5:** Use PKI for websites

Next Week

- TCP/IP Attacks
 - SYN Flooding
 - ARP cache Poisoning
 - ICMP redirect